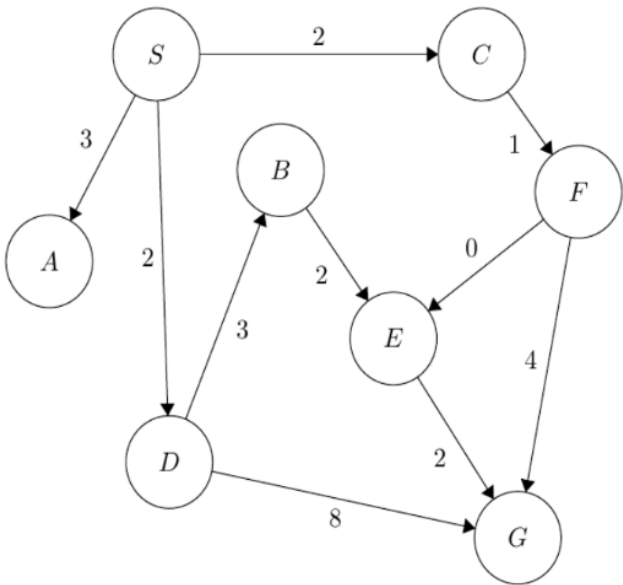


BÀI TẬP TÌM KIẾM TRONG AI

Bài 1. Các thuật toán tìm kiếm

Cho đồ thị có trạng thái bắt đầu S và trạng thái đích G như hình dưới đây.



Hình 1. Đồ thị dùng cho các thuật toán tìm kiếm

Yêu cầu: Với mỗi thuật toán, hãy ghi thứ tự các nút được đưa vào explored set, đường đi được trả về và tổng chi phí của đường đi. Nếu có hòa, phá hòa theo thứ tự alphabet của trạng thái cuối cùng trong đường đi.

1.1 Breadth-First Search (BFS)

Nội dung	Kết quả
Explored Set	S, A, C, D, B, F, G
Path	S → D → G
Total Cost	10

1.2 Depth-First Search (DFS)

Nội dung	Kết quả
Explored Set	S, A, C, F, E, G
Path	S → C → F → E → G
Total Cost	5

1.3 Iterative Deepening Search (IDS)

Giới hạn độ sâu	Các nút được duyệt
0	S
1	S, A, C, D
2	S, A, C, F, D, B, G
...	

Path cuối cùng: $S \rightarrow D \rightarrow G$

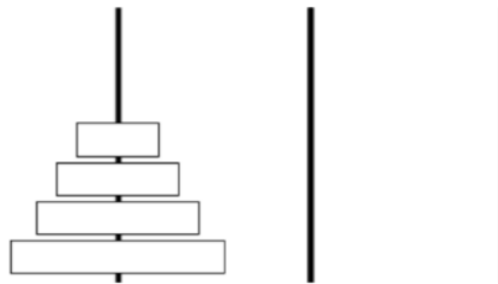
Total Cost: 10

Câu hỏi thảo luận: BFS và DFS khác nhau ở cấu trúc frontier và tính tối ưu như thế nào?

BFS sử dụng hàng đợi FIFO (First In First Out) cho frontier nên luôn mở rộng các nút có độ sâu nhỏ trước. Vì vậy, BFS tìm được đường đi có số bước ít nhất khi chi phí các bước bằng nhau. DFS sử dụng ngăn xếp LIFO (Last In First Out), ưu tiên đi sâu vào một nhánh trước khi quay lui. DFS không đảm bảo tìm được đường đi tối ưu vì có thể tìm thấy một lời giải sâu hơn trước một lời giải tốt hơn.

Bài 2. Tháp Hà Nội – Tower of Hanoi

Bài toán gồm n đĩa có kích thước khác nhau và ba cọc. Ban đầu các đĩa được xếp trên một cọc theo thứ tự từ lớn đến nhỏ. Mỗi lần chỉ được di chuyển một đĩa và không được đặt đĩa lớn lên trên đĩa nhỏ. Mục tiêu là chuyển toàn bộ đĩa sang cọc thứ ba.



Hình 2. Trạng thái ban đầu của bài toán Tháp Hà Nội

2.1 Biểu diễn trạng thái

“Hãy đề xuất cách biểu diễn một trạng thái của bài toán. Biểu diễn phải cho biết mỗi đĩa đang ở cọc nào và đủ để xác định các hành động hợp lệ.”

Biểu diễn trạng thái bằng một bộ $S = (p_1, p_2, \dots, p_n)$

Trong đó: $p_i \in \{1, 2, 3\}$, p_i cho biết đĩa thứ i hiện đang nằm ở cọc nào.

Quy ước đĩa 1 là đĩa nhỏ nhất, đĩa n là đĩa lớn nhất.

Ví dụ 4 đĩa: $S = (1, 1, 2, 3) \Rightarrow$ đĩa 1 ở cọc 1, đĩa 2 ở cọc 1, đĩa 3 ở cọc 2, đĩa 4 ở cọc 3.

Cách biểu diễn này đủ để xác định đĩa nào đang nằm trên cùng của từng cọc, từ đó xác định các hành động hợp lệ.

2.2 Kích thước không gian trạng thái

“Với n đĩa, hãy xác định số trạng thái có thể có theo cách biểu diễn của bạn và giải thích cách suy luận.”

Mỗi đĩa có thể nằm trên 1 trong 3 cọc. Vì có n đĩa nên số cách phân bố các đĩa là: 3^n

Suy ra kích thước không gian trạng thái của bài toán là 3^n .

2.3 Trạng thái bắt đầu

“Hãy viết trạng thái khởi đầu S_0 theo cách biểu diễn đã chọn.”

Ban đầu toàn bộ n đĩa đều nằm ở cọc thứ nhất $\Rightarrow$ trạng thái bắt đầu là $S_0 = (1, 1, \dots, 1)$

Với 4 đĩa $\Rightarrow S_0 = (1, 1, 1, 1)$

2.4 Các hành động hợp lệ

“Từ một trạng thái bất kỳ, hãy mô tả tập Actions(s). Một hành động hợp lệ phải thỏa mãn các điều kiện nào?”

Một hành động có thể biểu diễn là $Move(i, A, B)$ nghĩa là chuyển đĩa i từ cọc A sang B và chỉ hợp lệ khi thỏa mãn: **đĩa được chọn đang nằm trên cùng của cọc nguồn và cọc đích đang trống hoặc có đĩa trên cùng lớn hơn đĩa cần di chuyển. Không được phép đặt một đĩa lớn lên trên một đĩa nhỏ.**

2.5 Goal Test

“Hãy mô tả hàm goalTest(state): khi nào hàm trả về True?”

goalTest(state):

if mọi đĩa đều ở cọc 3:

return True

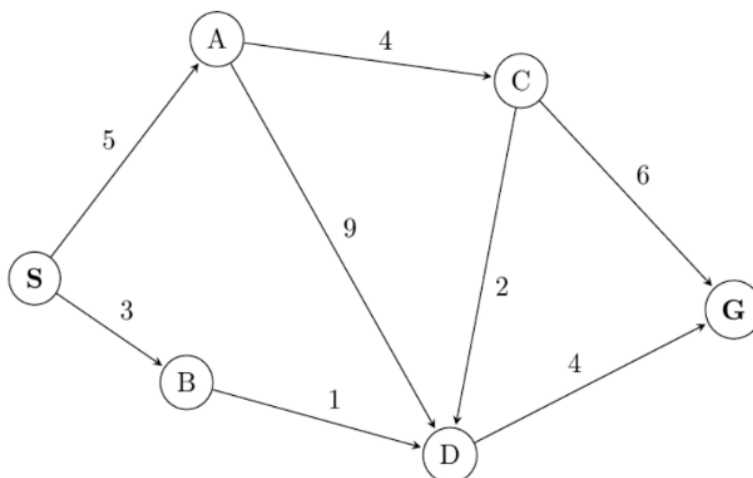
else:

return False

Hàm trả về True khi tất cả các đĩa đều nằm trên cọc thứ ba. Trạng thái đích là $G = (3, 3, 3, 3)$. Nếu tồn tại ít nhất 1 đĩa không nằm ở cọc 3 thì hàm trả về False.

Bài 3. Thiết kế và phân tích Heuristic

Xét đồ thị dưới đây. Hãy thiết kế heuristic và phân tích ảnh hưởng của heuristic đến thuật toán A*.



Hình 3. Đồ thị dùng để thiết kế heuristic

Công thức A*: $f(n) = g(n) + h(n)$, trong đó $g(n)$ là chi phí thực tế từ S đến n và $h(n)$ là chi phí ước lượng từ n đến đích.

3.1 Thiết kế heuristic admissible và consistent

Điền giá trị $h(n)$ của từng nút sao cho heuristic phù hợp với yêu cầu của nhóm.

Node	$h(n)$
S	8
A	10
B	5
C	6
D	4
G	0

3.2 Chạy A* với heuristic đã thiết kế

Explored Set: $S \rightarrow B \rightarrow D \rightarrow G$

Path: $S \rightarrow B \rightarrow D \rightarrow G$

Total Cost: $3 + 1 + 4 = 8$

3.3 Heuristic admissible nhưng không consistent

Đề xuất một bộ giá trị $h(n)$ admissible nhưng không consistent. Chỉ ra ít nhất một cạnh vi phạm điều kiện $h(n) \leq c(n, n') + h(n')$

Chọn $h(S) = 8, h(A) = 10, h(B) = 5, h(C) = 6, h(D) = 0, h(G) = 0$. Bộ heuristic này vẫn admissible vì không có giá trị nào lớn hơn chi phí tối ưu thực sự từ nút đó đến G. Tuy nhiên nó không consistent. Xét cạnh $B \rightarrow D$ có $cost = 1$:

$$h(B) = 5 > 1 + h(D) = 1$$

nên vi phạm điều kiện consistency.

3.4 Câu hỏi mở rộng

Giải thích vì sao một heuristic consistent thì cũng admissible, giả sử $h(G) = 0$

Nếu heuristic consistent thì với mọi cạnh $n \rightarrow n'$:

$$h(n) \leq c(n, n') + h(n')$$

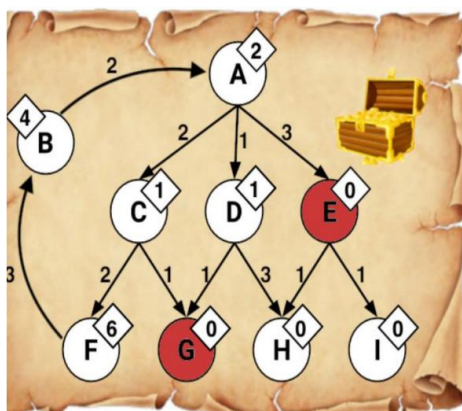
Áp dụng liên tiếp trên một đường đi từ n đến G và vì $h(G) = 0$ ta có:

$$h(n) \leq cost(n \rightarrow G)$$

Do đó $h(n)$ không bao giờ lớn hơn chi phí tối ưu thực sự đến G. Vì vậy mọi heuristic consistent đều admissible.

Bài 4. Treasure Hunting (Tìm kho báu)

A là đảo xuất phát. Các nút màu đỏ là các đảo có kho báu; chỉ cần tới một trong các đảo đó. Mũi tên biểu diễn hành động có hướng; số trên cạnh là action cost. Giá trị trong hình tròn là heuristic ước lượng chi phí tối thiểu từ nút hiện tại tới một kho báu.



Hình 4. Bản đồ tìm kho báu (giữ nguyên hình gốc)

Quy tắc phá hòa: khi lấy phần tử khỏi frontier, nếu hòa thì chọn theo thứ tự alphabet.

4.1 Depth-First Search

Explored Set: A, C, F, B, G

Path returned: $A \rightarrow C \rightarrow G$

Cost: 3

4.2 Breadth-First Search

Explored Set: A, C, D, E

Path returned: $A \rightarrow E$

Cost: 3

4.3 Uniform-Cost Search

Explored Set: A, D, C, G

Path returned: $A \rightarrow D \rightarrow G$

Cost: $1 + 1 = 2$

4.4 Greedy Search

Explored Set: A, E

Path returned: $A \rightarrow E$

Cost: 3

4.5 A* Search

Explored Set: A, D, G

Path returned: $A \rightarrow D \rightarrow G$

Cost: $1 + 1 = 2$

Bảng tổng hợp so sánh:

Thuật toán	Tiêu chí chọn node	Path	Cost
DFS	Độ sâu	$A \rightarrow C \rightarrow G$	3
BFS	Độ sâu nhỏ nhất	$A \rightarrow E$	3
UCS	$g(n)$	$A \rightarrow D \rightarrow G$	2
Greedy	$h(n)$	$A \rightarrow E$	3
A*	$g(n)+h(n)$	$A \rightarrow D \rightarrow G$	2

Bài 5. Câu hỏi Đúng/Sai

Với mỗi phát biểu, hãy ghi Đúng hoặc Sai và giải thích ngắn gọn. Nếu sai, nên đưa ra phản ví dụ.

Câu 1. Depth-first search luôn mở rộng ít nhất nhiều node bằng A* sử dụng một admissible heuristic.

Sai. DFS và A* có chiến lược mở rộng node hoàn toàn khác nhau. DFS đi sâu theo một nhánh còn A* chọn node dựa trên $f(n) = g(n) + h(n)$. Không có đảm bảo rằng DFS luôn mở rộng ít nhất hay nhiều nhất so với A*. Tùy cấu trúc đồ thị, DFS có thể tình cờ đi đúng nhánh và tìm goal rất nhanh, trong khi A* vẫn phải mở rộng thêm một số node khác.

Câu 2. Giả sử một quân xe trong cờ vua có thể đi bất kỳ số ô theo hàng hoặc cột trong một nước đi và không thể nhảy qua quân khác. Manhattan distance là một admissible heuristic cho số nước đi nhỏ nhất từ ô A đến ô B.

Sai. Manhattan distance: $h = |x_1 - x_2| + |y_1 - y_2|$ đếm số ô chênh lệch nhưng quân xe có thể đi nhiều ô trong một nước.

Ví dụ từ: $A = (1,1)$ đến $B = (1,8)$ thì $h = |1 - 1| + |8 - 1| = 7$ nhưng quân xe chỉ cần 1 nước đi.

Vậy $h = 7 > h^* = 1$ nên heuristic này đánh giá quá cao chi phí thật và không admissible.

Câu 3. Euclidean distance là một admissible heuristic cho bài toán tìm đường của Pacman.

Đúng, với mô hình Pacman thông thường di chuyển từng ô theo 4 hướng. Khoảng cách Euclidean là: $h(n) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$

Trong lưới mà Pacman di chuyển theo Bắc, Nam, Đông, Tây, chi phí đường đi thực tế không thể nhỏ hơn khoảng cách đường thẳng giữa hai điểm. Do đó: $h(n) \leq h^*(n)$ nên Euclidean distance là admissible.

Câu 4. Tổng của nhiều admissible heuristics vẫn luôn là một admissible heuristic.

Sai. Giả sử $h_1(n) \leq h^*(n)$ và $h_2(n) \leq h^*(n)$ không có nghĩa là: $h_1(n) + h_2(n) \leq h^*(n)$.

Ví dụ chi phí thật: $h^*(n) = 10$ và $h_1(n) = 6, h_2(n) = 6$. Cả hai đều admissible vì $6 \leq 10$ nhưng $h_1 + h_2 = 12 > 10$ nên tổng không admissible.

Câu 5. Admissibility của một heuristic cho A* kéo theo consistency.

Sai. Admissible chỉ yêu cầu $h(n) \leq h^*(n)$ tức heuristic không được lớn hơn chi phí tối ưu thật tới goal. Consistency mạnh hơn, yêu cầu với mọi cạnh: $h(n) \leq c(n, n') + h(n')$.

Một heuristic có thể không đánh giá quá cao chi phí đến goal nhưng vẫn vi phạm bất đẳng thức trên ở một cạnh nào đó.

Ví dụ: $h(B) = 5, h(D) = 0$ và $c(B, D) = 1$. Ta có: $5 > 1 + 0$ nên không consistent, dù heuristic vẫn admissible.

Câu 6. A* với graph search luôn tối ưu nếu heuristic admissible.

Sai. Với A* graph search, admissible heuristic chưa đủ trong cách triển khai chuẩn có explored set mà không mở lại node. Để đảm bảo tối ưu 1 cách an toàn thì heuristic thường cần consistent. Nếu heuristic chỉ admissible nhưng không consistent, một node có thể bị đưa vào explored bằng 1 đường đi chưa tối ưu. Sau đó nếu tìm được đường tốt hơn tới node đó nhưng thuật toán không reopen node, A* có thể trả về lời giải không tối ưu.

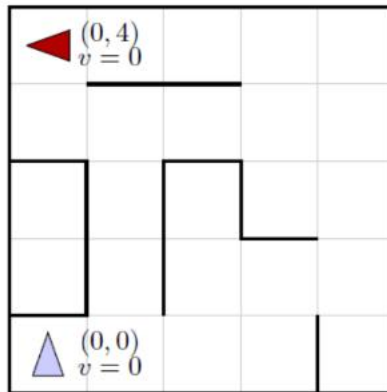
Tổng kết nhanh

Nhóm	Thuật toán	Hàm/tiêu chí chính
Uniformed Search	BFS / DFS / IDS	Không sử dụng heuristic
Cost-based Search	UCS	$g(n)$
Informed Search	Greedy	$h(n)$
Informed Search	A*	$g(n)+h(n)$

Ghi nhớ: A* dùng $f(n) = g(n) + h(n)$

BÀI TẬP TÌM KIẾM VÀ HEURISTIC TRONG AI

Câu 1. Tìm kiếm và heuristic



Một tác tử dạng xe cần thoát khỏi mê cung như hình trên. Tác tử có hướng và tại mọi thời điểm luôn quay về một trong bốn hướng $d \in \{\text{Bắc, Nam, Đông, Tây}\}$. Trong mỗi bước, tác tử có thể tiến về phía trước với vận tốc có thể điều chỉnh hoặc thực hiện thao tác rẽ.

Các hành động di chuyển gồm tăng tốc, giữ tốc độ và giảm tốc. Sau hành động, tác tử di chuyển một số ô bằng vận tốc mới v' . Gọi v là vận tốc hiện tại và v' là vận tốc sau điều chỉnh:

- Tăng tốc (Faster): $v' = v + 1$
- Giảm tốc (Slower): $v' = v - 1$
- Giữ tốc độ (Maintain): $v' = v$

Các hành động rẽ gồm trái và phải, làm thay đổi hướng 90° . Chỉ được phép rẽ khi vận tốc bằng 0; sau khi rẽ vận tốc vẫn bằng 0.

- Rẽ trái: quay 90° ngược chiều kim đồng hồ.
- Rẽ phải: quay 90° theo chiều kim đồng hồ.

Ví dụ: nếu tác tử ở $(0,0)$, quay về Bắc, $v = 0$ và cần đến $(2,0)$, quay về Đông, $v = 0$ thì một chuỗi hành động hợp lệ là: rẽ phải, tăng tốc, giữ tốc độ, giảm tốc.

Các hành động không hợp lệ gồm:

- Hành động làm tác tử va vào tường.
- Làm $v < 0$ hoặc $v > V_{\max}$.
- Giữ tốc độ khi $v = 0$.
- Rẽ khi $v \neq 0$.

Mục tiêu là tìm kế hoạch đưa xe dừng hẳn ($v = 0$) tại ô thoát, đúng hướng đích, với số hành động ít nhất. Chi phí đường đi được tính bằng số hành động đã thực hiện.

a/ Giả sử tác tử đi theo nhánh ngoài cùng bên trái (đi qua ô $(1,2)$), từ trạng thái bắt đầu $(0,0)$, hướng Bắc đến trạng thái đích $(0,4)$, hướng Tây. Hãy viết chuỗi hành động ngắn nhất.

Yêu cầu đi từ: $(0,0)$, Bắc, $v = 0$ đến $(0,4)$, Tây, $v = 0$ và đi theo nhánh ngoài cùng bên trái tức là đi qua ô $(1,2) \Rightarrow$

Đường đi theo nhánh này là $(0,0) \rightarrow (1,0) \rightarrow (1,1) \rightarrow (1,2) \rightarrow (1,3) \rightarrow (0,3) \rightarrow (0,4)$

Do xe chỉ được rẽ khi $v = 0$ nên trước mỗi lần đổi hướng phải giảm tốc về 0.

Đoạn 1: từ (0,0) sang (1,0)

Ban đầu hướng Bắc nên phải rẽ phải:

1. Turn Right
2. Faster $\rightarrow v=1$, đi 1 ô
3. Slower $\rightarrow v=0$

Đến: (1,0), Đông, $v=0$

Sau đó rẽ trái để hướng Bắc.

4. Turn Left

Đoạn 2: đi từ (1,0) đến (1,3)

Cần đi thẳng 3 ô:

5. Faster $\rightarrow v=1$, đi 1 ô $\rightarrow (1,1)$
6. Maintain $\rightarrow v=1$, đi 1 ô $\rightarrow (1,2)$
7. Maintain $\rightarrow v=1$, đi 1 ô $\rightarrow (1,3)$
8. Slower $\rightarrow v=0$

Đoạn 3: từ (1,3) sang (0,3)

9. Turn Left $\rightarrow$ hướng Tây
10. Faster $\rightarrow$ đi 1 ô
11. Slower $\rightarrow v=0$

Đoạn 4: từ (0,3) đến (0,4)

12. Turn Right $\rightarrow$ hướng Bắc
13. Faster $\rightarrow$ đi 1 ô
14. Slower $\rightarrow v=0$

Đến ô đích nhưng đang hướng Bắc, nhưng đề yêu cầu hướng Tây $\Rightarrow$

15. Turn Left

Trạng thái cuối: (0,4), Tây, $v=0$

Chuỗi hành động ngắn nhất theo nhánh ngoài cùng bên trái là: Turn Right $\rightarrow$ Faster $\rightarrow$ Slower $\rightarrow$ Turn Left $\rightarrow$ Faster $\rightarrow$ Maintain $\rightarrow$ Maintain $\rightarrow$ Slower $\rightarrow$ Turn Left $\rightarrow$ Faster $\rightarrow$ Slower $\rightarrow$ Turn Right $\rightarrow$ Faster $\rightarrow$ Slower $\rightarrow$ Turn Left (15 hành động)

b/ Nếu lưới có kích thước $M \times N$ và vận tốc tối đa là V_{max} , kích thước không gian trạng thái là bao nhiêu? Giả sử mọi cấu hình đều có thể đạt được từ trạng thái bắt đầu.

Đề cho lưới kích thước: $M \times N$ và vận tốc: $0 \leq v \leq V_{max}$

Một trạng thái gồm: (x, y, d, v)

Ta có:

- Vị trí: $M \times N$ khả năng
- Hướng: 4 khả năng
- Vận tốc: $V_{max} + 1$ khả năng, gồm $0, 1, 2, \dots, V_{max}$

Vậy số trạng thái: $M \times N \times 4 \times (V_{max} + 1)$ hay $4MN(V_{max} + 1)$

c/ Một “nút con” của trạng thái s là trạng thái s' có thể đạt được bằng một hành động hợp lệ từ s . Có thể tồn tại trạng thái không có nút con hay không? Nếu có, hãy cho ví dụ; nếu không, hãy giải thích.

☐ Có ☐ Không

Không. Vì nếu $v = 0$ thì xe luôn có thể Turn Left, Turn Right vì việc rẽ không làm xe di chuyển sang ô khác. Nếu $v > 0$ thì ít nhất trong các hành động điều chỉnh vận tốc có thể tồn tại hành động hợp lệ trong một trạng thái hợp lệ; đặc biệt xe có thể giảm tốc nếu cần. Quan trọng nhất, một trạng thái đứng yên $v = 0$ không thể là dead-end vì luôn có thể quay tại chỗ.

d/ Hệ số phân nhánh lớn nhất của bài toán là bao nhiêu? Hãy cho một trạng thái ví dụ $(x, y, hướng, vận tốc)$ đạt hệ số phân nhánh này và liệt kê các hành động có thể thực hiện.

Đề có tất cả 5 loại hành động nhưng chúng không thể cùng hợp lệ.

Khi: $v = 0$

- Faster: có thể hợp lệ
- Slower: không hợp lệ vì $v' < 0$
- Maintain: không hợp lệ khi $v = 0$
- Turn Left: hợp lệ
- Turn Right: hợp lệ

Tối đa: 3 hành động

Khi: $v > 0$ không được rẽ, nhưng có thể có: Faster, Maintain, Slower $\rightarrow$ cũng tối đa 3 hành động.

Do đó: $b_{max} = 3$

Ví dụ: Xét $s = (1, 0, \text{Bắc}, 0)$ với ô phía Bắc không bị tường chắn.

Ba hành động có thể thực hiện là:

1. Faster
2. Turn Left
3. Turn Right

e/ Khoảng cách Manhattan từ vị trí hiện tại của tác tử đến vị trí cửa ra có phải là heuristic chấp nhận được không? Nếu không, hãy cho một trạng thái phản ví dụ, nêu giá trị heuristic và chi phí thực tế tối ưu đến đích.

☐ Có ☐ Không

Không. Manhattan distance đo số ô cách đích, trong khi một hành động có thể di chuyển nhiều hơn một ô khi vận tốc lớn hơn 1. Ví dụ nếu xe cách đích 5 ô, đang hướng về đích với $v = 1$, nó có thể đến đích và dừng lại trong 4 hành động: Faster, Maintain, Slower, Slower. Khi đó $h = 5 > h^* = 4$, vì vậy Manhattan distance không admissible.

f/ Heuristic $h = \text{khoảng cách Manhattan} / V_{max}$ có phải là admissible không? Giải thích. Nếu không, hãy cho phản ví dụ tương tự câu (e).

☐ Có ☐ Không

Có. Trong mỗi hành động, tác tử chỉ có thể di chuyển tối đa V_{max} ô. Vì vậy để vượt khoảng cách Manhattan D_M , cần ít nhất D_M/V_{max} hành động. Việc rẽ, giảm tốc và tránh tường chỉ có thể làm tăng chi phí thực tế. Do đó: $h(s) = \frac{D_M}{V_{max}} \leq h^*(s)$ nên heuristic này admissible.

g/ Nếu dùng heuristic không admissible trong A* Tree Search, tính đầy đủ của thuật toán có thể thay đổi không? Giả sử đồ thị hữu hạn và heuristic không âm.

☐ Có ☐ Không

Không. Với đồ thị hữu hạn, heuristic không âm và chi phí hành động dương, việc heuristic không admissible chỉ thay đổi thứ tự ưu tiên mở rộng các node, không làm mất tính đầy đủ của A* Tree Search. Nếu có lời giải, thuật toán vẫn có thể tìm được lời giải.

h/ Nếu dùng heuristic không admissible trong A* Tree Search, tính tối ưu của thuật toán có thể thay đổi không? Giả sử đồ thị hữu hạn và heuristic không âm.

☐ Có ☐ Không

Có. Heuristic không admissible có thể đánh giá quá cao chi phí của nhánh chứa lời giải tối ưu. Khi đó A* có thể ưu tiên một nhánh khác và trả về goal có chi phí lớn hơn. Vì vậy A* không còn được đảm bảo tối ưu.

i/ Những lý do nào sau đây có thể hợp lý khi dùng heuristic không admissible thay vì admissible? Chọn tất cả phương án phù hợp.

☐ Heuristic không admissible có thể dễ tính hơn, giúp giảm thời gian tính heuristic.

Chọn vì một heuristic đơn giản có thể được tính nhanh hơn, mặc dù đôi lúc đánh giá quá cao.

☐ Heuristic không admissible có thể gần chi phí thực hơn (dù đôi khi đánh giá cao), nhờ đó có thể mở ít nút hơn.

Chọn vì đây là một lý do thực tế phổ biến. Chấp nhận mất bảo đảm tối ưu có thể giúp tìm lời giải nhanh hơn.

☐ Heuristic không admissible vẫn luôn tìm được đường đi tối ưu khi mọi chi phí thực không âm.

Không chọn vì heuristic không admissible có thể làm A* trả về lời giải không tối ưu, như ở câu h.

☐ Heuristic không admissible có thể được dùng để làm thuật toán hoàn toàn không xét một phần của đồ thị.

Chọn vì có thể gán giá trị heuristic rất lớn cho những khu vực không mong muốn, khiến thuật toán ưu tiên mạnh các khu vực khác và trong thực tế có thể kết thúc trước khi mở rộng các vùng đó. Đổi lại, ta mất bảo đảm tìm lời giải tối ưu.

Câu 2. Các nút tìm kiếm

Xét tree search (không dùng tập explored) cho một bài toán tìm kiếm bất kỳ có hệ số phân nhánh tối đa b. Mỗi nút tìm kiếm n có chi phí tích lũy $g(n)$, heuristic admissible $h(n)$ và độ sâu $d(n)$. Gọi n_c là nút đích có chi phí nhỏ nhất và n_s là nút đích nông nhất.

Với mỗi trường hợp, hãy viết một bất đẳng thức mô tả tập các nút được mở rộng trước khi thuật toán kết thúc. Không cần xử lý trường hợp hòa (tie). Nếu không có nút nào thỏa, ghi “không có”.

a/ BFS

Bất đẳng thức:

Với BFS, mọi node có độ sâu nhỏ hơn độ sâu của goal nông nhất đều được mở rộng trước khi thuật toán kết thúc.

$$d(n) < d(n_s)$$

b/ UCS

Bất đẳng thức:

UCS mở rộng các node theo thứ tự tăng dần của chi phí tích lũy $g(n)$. Vì vậy tất cả node có chi phí nhỏ hơn chi phí của goal tối ưu sẽ được mở rộng trước:

$$g(n) < g(n_c)$$

c/ A* Search, giả sử h là heuristic nhất quán

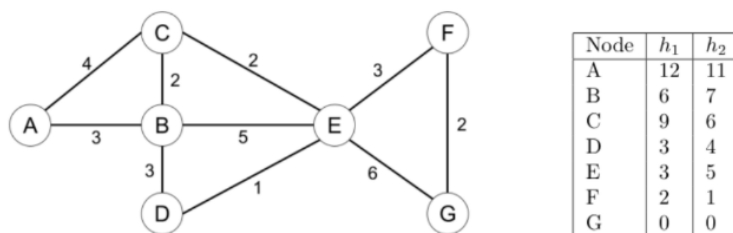
A* chọn node theo: $f(n) = g(n) + h(n)$

Với goal tối ưu n_c , ta có $h(n_c) = 0$ nên: $f(n_c) = g(n_c)$

Vì vậy các node chắc chắn được mở rộng trước khi thuật toán kết thúc thỏa:

$$g(n) + h(n) < g(n_c)$$

Câu 3. Tìm kiếm trên đồ thị



Nút	h1	h2
A	12	11
B	6	7
C	9	6
D	3	4

E	3	5
F	2	1
G	0	0

Xét đồ thị vô hướng như hình. A là trạng thái bắt đầu và G là trạng thái đích. Chi phí của mỗi cạnh được ghi trên đồ thị. Bảng trên cung cấp hai heuristic h1 và h2. Khi có nhiều nút cùng ưu tiên, kết quả có thể phụ thuộc quy tắc phá hòa (tie-breaking).

a/ Với mỗi chiến lược tìm kiếm đồ thị dưới đây, hãy đánh dấu X vào mọi đường đi mà thuật toán có thể trả về. Nếu kết quả không nằm trong các đường đi liệt kê, ghi đường đi đúng ở cột “Khác”.

Thuật toán	A-C-E-G	A-C-E-F-G	A-B-D-E-F-G	Khác
UCS		x		
Greedy với h1				A-B-E-G
Greedy với h2	x			
A* với h1			x	
A* với h2		x		

b/ Chi phí của đường đi tối ưu từ A đến G theo UCS là bao nhiêu?

$A \rightarrow C \rightarrow E \rightarrow F \rightarrow G$ có $4 + 2 + 3 + 2 = 11$

c/ Heuristic h1 có admissible không? Có consistent không?

Admissible: Không, Consistent: Không

h1 không admissible vì $h_1(A) = 12 > h^*(A) = 11$

h1 cũng không consistent vì trên cạnh $A - B$: $h_1(A) = 12 > 3 + h_1(B) = 9$

d/ Heuristic h2 có admissible không? Có consistent không?

Admissible: Có, Consistent: Không

h2 admissible vì tất cả giá trị $h_2(n)$ đều không vượt quá chi phí tối ưu thực tế $h^*(n)$. Tuy nhiên h2 không consistent. Ví dụ trên cạnh $A - B$: $h_2(A) = 11 > 3 + h_2(B) = 10$

Câu 4. Trắc nghiệm và trả lời ngắn về tìm kiếm

a/ Đánh dấu Đúng/Sai. Giả sử mọi hành động có chi phí ít nhất ϵ , với $\epsilon > 0$, và mọi heuristic được dùng đều consistent.

Depth-First Tree Search trên một đồ thị hữu hạn luôn được đảm bảo là complete.

Sai. Tree Search không loại bỏ trạng thái lặp. Nếu đồ thị có chu trình, DFS có thể đi vô hạn theo một nhánh lặp và không tìm được goal dù goal tồn tại.

Breadth-First Tree Search trên một đồ thị hữu hạn luôn được đảm bảo là complete.

Đúng. BFS mở rộng các node theo từng mức độ sâu. Nếu goal tồn tại ở một độ sâu hữu hạn và hệ số phân nhánh hữu hạn, BFS sẽ tìm được goal.

Iterative Deepening Tree Search trên một đồ thị hữu hạn luôn được đảm bảo là complete.

Đúng. IDS tăng dần giới hạn độ sâu. Nếu goal tồn tại ở độ sâu hữu hạn, thuật toán sẽ tìm thấy khi giới hạn đạt tới độ sâu đó.

Với mọi đồ thị không có chu trình, Graph Search luôn có frontier lớn hơn Tree Search.

Sai. Graph Search loại bỏ các trạng thái đã được gặp trước đó, trong khi Tree Search có thể chứa nhiều node tương ứng cùng một trạng thái. Vì vậy frontier của Graph Search không nhất thiết lớn hơn Tree Search.

Iterative Deepening Graph Search có độ phức tạp thời gian của BFS và độ phức tạp bộ nhớ của DFS.

Sai. Tính chất thời gian gần BFS và bộ nhớ gần DFS là của Iterative Deepening Tree Search. Với Graph Search, cần lưu explored set nên bộ nhớ có thể tăng đáng kể.

Nếu $h_1(s)$ và $h_2(s)$ đều là heuristic consistent thì $\min(h_1(s), h_2(s))$ cũng nhất thiết consistent.

Đúng. Nếu h_1 và h_2 đều thỏa điều kiện consistency trên mọi cạnh thì hàm lấy giá trị nhỏ hơn của hai heuristic cũng vẫn thỏa điều kiện đó.

b/ Xét đồ thị trạng thái dưới đây, S là bắt đầu và G là đích. Hãy điền các giá trị heuristic sao cho heuristic admissible nhưng không consistent.

Trạng thái	$h(s)$
S	4
A	0
C	2
G	0

Chọn: $h(S) = 4, h(A) = 0, h(C) = 2, h(G) = 0$

Bộ heuristic này admissible vì không giá trị nào vượt quá chi phí tối ưu thực sự tới G. Tuy nhiên nó không consistent vì trên cạnh $S - A$ có cost bằng 1:

$$h(S) = 4 > 1 + h(A) = 1$$

nên vi phạm điều kiện consistency.